Data Structures and Algorithms Lab
Assignment 5

Experiment 5 (Tree): To demonstrate the creation of a binary tree using arrays/linked lists and working with tree traversal.

Instruction: Make a pdf file of the program code in C and its output.

Name:
SAP ID:
Batch:
Link of your GitHub Account:
Course: B.Tech. CSE (Sem II)
Subject: DSA Lab
Date: 07 April 2026
Note: Do all the implementations in C programming.

1. Write a C program to create a binary tree using an array. The program should support the following operations:
   (i) Insert elements into the tree.
   (ii) Display the tree elements.
   (iii) Display the parent, left child, and right child of a given node.

Assignment5.c

Assignment5.c > main()

```c
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
int tree[MAX];
int n = 0;
void insert(int val) {
    if (n >= MAX) {
        printf("Tree is full!\n");
        return;
    }
    tree[n] = val;
    n++;
}
void display() {
    printf("\nTree elements (index: value):\n");
    for (int i = 0; i < n; i++) {
        printf("Index %d: %d\n", i, tree[i]);
    }
}
void nodeInfo(int index) {
    if (index < 0 || index >= n) {
        printf("Invalid index!\n");
        return;
    }
    printf("\nNode at index %d: %d\n", index, tree[index]);
    int parent = (index - 1) / 2;
    if (index == 0)
        printf("Parent: None (root node)\n");
    else
        printf("Parent: %d (index %d)\n", tree[parent], parent);
    int left = 2 * index + 1;
    if (left < n)
        printf("Left Child: %d (index %d)\n", tree[left], left);
    else
        printf("Left Child: None\n");
    int right = 2 * index + 2;
    if (right < n)
        printf("Right Child: %d (index %d)\n", tree[right], right);
    else
        printf("Right Child: None\n");
}
int main() {
    int choice, val, idx;
    do {
        printf("\n--- Binary Tree (Array) ---\n");
        printf("1. Insert\n2. Display\n3. Node Info\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &val);
                insert(val);
```

```c
int main() {
    int choice, val, idx;
    do {
        printf("\n--- Binary Tree (Array) ---\n");
        printf("1. Insert\n2. Display\n3. Node Info\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &val);
                insert(val);
                break;
            case 2:
                display();
                break;
            case 3:
                printf("Enter index: ");
                scanf("%d", &idx);
                nodeInfo(idx);
                break;
            case 4:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice!\n");
        }
    } while (choice != 4);
    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Assignment5

```
--- Binary Tree (Array) ---
1. Insert
2. Display
3. Node Info
4. Exit
Enter choice: 3
Enter index: 1
Invalid index!

--- Binary Tree (Array) ---
1. Insert
2. Display
3. Node Info
```

2. Write a C program to create a binary tree using linked list representation and perform various operations on it.
   (i) Insert nodes into the tree (use user input).
   (ii) Create separate functions to count.
      - Total number of nodes.
      - Number of leaf nodes (nodes with no children).
      - Height of the binary tree.

Assignment5.c

Assignment5.c > insert(Node *, int)

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;
Node* newNode(int val) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->left = n->right = NULL;
    return n;
}
Node* insert(Node* root, int val) {
    Node* node = newNode(val);
    if (!root) return node;
    Node* queue[100];
    int front = 0, rear = 0;
    queue[rear++] = root;

    while (front < rear) {
        Node* curr = queue[front++];
        if (!curr->left) { curr->left = node; return root; }
        else queue[rear++] = curr->left;
        if (!curr->right) { curr->right = node; return root; }
        else queue[rear++] = curr->right;
    }
    return root;
}
int countNodes(Node* root) {
    if (!root) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}
int countLeaves(Node* root) {
    if (!root) return 0;
    if (!root->left && !root->right) return 1;
    return countLeaves(root->left) + countLeaves(root->right);
}
int height(Node* root) {
    if (!root) return 0;
    int lh = height(root->left);
    int rh = height(root->right);
    return 1 + (lh > rh ? lh : rh);
}
void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}
```

```c
int main() {
    Node* root = NULL;
    int choice, val;
    do {
        printf("\n--- Binary Tree (Linked List) ---\n");
        printf("1. Insert\n2. Count Nodes\n3. Count Leaves\n4. Height\n5. Inorder\n6. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &val);
                root = insert(root, val);
                printf("Inserted %d\n", val);
                break;
            case 2:
                printf("Total nodes: %d\n", countNodes(root));
                break;
            case 3:
                printf("Leaf nodes: %d\n", countLeaves(root));
                break;
            case 4:
                printf("Height of tree: %d\n", height(root));
                break;
            case 5:
                printf("Inorder: ");
                inorder(root);
                printf("\n");
                break;
            case 6:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice!\n");
        }
    } while (choice != 6);
    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Total nodes: 2

--- Binary Tree (Linked List) ---
1. Insert
2. Count Nodes
3. Count Leaves
4. Height
5. Inorder
6. Exit
Enter choice: 4
Height of tree: 2

--- Binary Tree (Linked List) ---
```

3. Write a C program to implement recursive tree traversals on a binary tree. Your program should perform the following traversal functions:
   (i) Inorder traversal (Left → Root → Right).
   (ii) Preorder traversal (Root → Left → Right).
   (iii) Postorder traversal (Left → Right → Root).

Assignment5.c

Assignment5.c > insert(Node *, int)

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;
Node* newNode(int val) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->left = n->right = NULL;
    return n;
}
Node* insert(Node* root, int val) {
    Node* node = newNode(val);
    if (!root) return node;
    Node* queue[100];
    int front = 0, rear = 0;
    queue[rear++] = root;

    while (front < rear) {
        Node* curr = queue[front++];
        if (!curr->left) { curr->left = node; return root; }
        else queue[rear++] = curr->left;
        if (!curr->right) { curr->right = node; return root; }
        else queue[rear++] = curr->right;
    }
    return root;
}
int countNodes(Node* root) {
    if (!root) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}
int countLeaves(Node* root) {
    if (!root) return 0;
    if (!root->left && !root->right) return 1;
    return countLeaves(root->left) + countLeaves(root->right);
}
int height(Node* root) {
    if (!root) return 0;
    int lh = height(root->left);
    int rh = height(root->right);
    return 1 + (lh > rh ? lh : rh);
}
void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}
```

```c
int main() {
    Node* root = NULL;
    int choice, val;
    do {
        printf("\n--- Binary Tree (Linked List) ---\n");
        printf("1. Insert\n2. Count Nodes\n3. Count Leaves\n4. Height\n5. Inorder\n6. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &val);
                root = insert(root, val);
                printf("Inserted %d\n", val);
                break;
            case 2:
                printf("Total nodes: %d\n", countNodes(root));
                break;
            case 3:
                printf("Leaf nodes: %d\n", countLeaves(root));
                break;
            case 4:
                printf("Height of tree: %d\n", height(root));
                break;
            case 5:
                printf("Inorder: ");
                inorder(root);
                printf("\n");
                break;
            case 6:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice!\n");
        }
    } while (choice != 6);
    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) chaitanyasharma@Chaitanyas-MacBook-Air Assignment 5 % ./Assignment5

--- Binary Tree (Linked List) ---
1. Insert
2. Count Nodes
3. Count Leaves
4. Height
5. Inorder
6. Exit
Enter choice: 4
Height of tree: 0
```

4. Write a C program to perform Level Order Traversal on a binary tree using a queue.

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;
Node* newNode(int val) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->left = n->right = NULL;
    return n;
}
Node* insert(Node* root, int val) {
    Node* node = newNode(val);
    if (!root) return node;
    Node* queue[100];
    int front = 0, rear = 0;
    queue[rear++] = root;
    while (front < rear) {
        Node* curr = queue[front++];
        if (!curr->left) { curr->left = node; return root; }
        else queue[rear++] = curr->left;
        if (!curr->right) { curr->right = node; return root; }
        else queue[rear++] = curr->right;
    }
    return root;
}
void levelOrder(Node* root) {
    if (!root) return;
    Node* queue[100];
    int front = 0, rear = 0;
    queue[rear++] = root;
    printf("Level Order Traversal:\n");
    int level = 1;
    int currCount = 1, nextCount = 0;
    printf("Level %d: ", level);
    while (front < rear) {
        Node* curr = queue[front++];
        printf("%d ", curr->data);
        currCount--;
        if (curr->left)  { queue[rear++] = curr->left;  nextCount++; }
        if (curr->right) { queue[rear++] = curr->right; nextCount++; }

        if (currCount == 0 && front < rear) {
            level++;
            printf("\nLevel %d: ", level);
            currCount = nextCount;
            nextCount = 0;
        }
    }
    printf("\n");
}
int main() {
    Node* root = NULL;
    int n, val;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter %d values:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        root = insert(root, val);
    }
    levelOrder(root);
    return 0;
}
```

```
(base) chaitanyasharma@Chaitanyas-MacBook-Air Assignment 5 % ./Assignment5

--- Binary Tree (Linked List) ---
1. Insert
2. Count Nodes
3. Count Leaves
4. Height
5. Inorder
6. Exit
Enter choice: 5
Inorder:
```

5. Write a C program to implement non-recursive tree traversals of a binary tree using a stack. Your program should implement the following traversals using the stack:
   (i) Inorder traversal (Left → Root → Right).
   (ii) Preorder traversal (Root → Left → Right).

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;
typedef struct Stack {
    Node* data[100];
    int top;
} Stack;
void initStack(Stack* s) { s->top = -1; }
int isEmpty(Stack* s) { return s->top == -1; }
void push(Stack* s, Node* n) { s->data[++(s->top)] = n; }
Node* pop(Stack* s) { return s->data[(s->top)--]; }
Node* peek(Stack* s) { return s->data[s->top]; }

Node* newNode(int val) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->left = n->right = NULL;
    return n;
}
Node* insert(Node* root, int val) {
    Node* node = newNode(val);
    if (!root) return node;
    Node* queue[100];
    int front = 0, rear = 0;
    queue[rear++] = root;
    while (front < rear) {
        Node* curr = queue[front++];
        if (!curr->left) { curr->left = node; return root; }
        else queue[rear++] = curr->left;
        if (!curr->right) { curr->right = node; return root; }
        else queue[rear++] = curr->right;
    }
    return root;
}
void inorderIterative(Node* root) {
    Stack s;
    initStack(&s);
    Node* curr = root;

    printf("Inorder (Non-Recursive): ");
    while (curr != NULL || !isEmpty(&s)) {
        while (curr != NULL) {
            push(&s, curr);
            curr = curr->left;
        }
        curr = pop(&s);
        printf("%d ", curr->data);
        curr = curr->right;
    }
```

```c
    printf("\n");
}
void preorderIterative(Node* root) {
    if (!root) return;
    Stack s;
    initStack(&s);
    push(&s, root);
    printf("Preorder (Non-Recursive): ");
    while (!isEmpty(&s)) {
        Node* curr = pop(&s);
        printf("%d ", curr->data);
        if (curr->right) push(&s, curr->right);
        if (curr->left)  push(&s, curr->left);
    }
    printf("\n");
}
int main() {
    Node* root = NULL;
    int n, val;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter %d values:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        root = insert(root, val);
    }
    inorderIterative(root);
    preorderIterative(root);
    return 0;
}
```

```
● (base) chaitanyasharma@Chaitanyas-MacBook-Air Assignment 5 % ./Assignment5
  Enter number of nodes: 3
  Enter 3 values:
  1 6 8
  Inorder (Non-Recursive): 6 1 8
  Preorder (Non-Recursive): 1 6 8
```